Part 3 Onboarding Guide

Returns and Rewards | POS Project (video game store)

Prepared for Sean on Oct 20, 2026 (Day 5 of the plan). Based on the Part Briefs (Oct 16), the current schema design document (updated Oct 16, evening) and the project plan.

1. At a glance

Item	Detail
Your part	Part 3: Returns and rewards. Customers request returns, support approves and refunds them, and the database restocks the items and reverses the loyalty points. Nothing blocks you: the PRODUCT_RETURN fix is already applied.
Branch	`part-3-returns`. Pull requests go to the lead, who merges to main. Vercel deploys from main only.
Folders you own	`db/returns/`, `api/returns/`, `api/rewards/`, `api/reports/rewards.js`, `src/pages/returns/`, and your own rewards report file in `src/pages/reports/`.
You build	2 triggers, SQL tests, 3 API areas (returns, rewards, rewards report), 5 screens, and a Jest test per endpoint.
Already past (today is Oct 20)	Foundation ready (Oct 17), schema freeze (end of Day 3, Oct 18) and the walking skeleton (end of Day 4, Oct 19). The schema is frozen: do not change tables without the lead's approval.
Still ahead	**Your `triggers.sql` and `test.sql` are due to the lead by the end of Day 6 (Wed Oct 21), which is tomorrow.** The lead runs the full chain through the app on Day 7 (Thu Oct 22). Feature freeze: Fri Oct 23. Test and bug bash: Oct 23 to 24. Submission: Oct 25.
Never cut	Returns with refund, restock and points reversal; the rewards report; login and roles (the lead's part). If you fall behind, tell the lead early: the lead takes over Part 3's refund third, after Part 2 (checkout) and Part 4's alert-to-restock flow.

2. Documents to read first

Document	How to use it
schema-current-design.md	**Source of truth** for tables, columns, triggers, decisions and the rebuild procedure. Newer than the conceptual schema PDF. It also contains the full text of the 5 built triggers.
POS Project: Part Briefs (PDF)	Your Part 3 page (page 6) plus the shared rules, the lead's deliverables, the timeline and the cut order.
Conceptual schema PDF	**Outdated. Ignore it for design.** It still has Register, Cashier, nullable Customer_id on transactions and cash payments, which were all removed.
Technology resources PDF	Tutorials for SQL Server, Node.js and React. Useful for learning, not for decisions.
triggers.sql, test_triggers.sql, drop.sql, schema file	SQL files from the lead (see section 4). `triggers.sql` is the style model for your two triggers: study TR_SALES_TRANSACTION_Complete (your refund trigger is its mirror) and TR_LOYALTY_TRANSACTION_Balance (it receives your negative loyalty row). `test_triggers.sql` is the model for your `db/returns/test.sql`.

3. Install the tools

These steps assume Windows (SQL Server Express runs on Windows). Run commands in PowerShell or the VS Code terminal (Ctrl+`). You already have VS Code, a SQL Server Express connection from VS Code and access to the team repository, so those are skipped. Git, Node.js, the project packages and the Vercel CLI are not installed yet, so do all of the steps below.

Step 1: Git

1. Download Git for Windows from https://git-scm.com/download/win and run the installer with the default options.
2. Close and reopen VS Code, then run `git --version`.
3. Set your identity (use your GitHub email):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 2: Node.js (LTS)

1. Download the **LTS** version from https://nodejs.org (not Current) and install with the defaults.
2. Reopen VS Code and check `node -v` and `npm -v`. Both must print a version.
3. If PowerShell says scripts are disabled when you run npm, run this once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Step 3: Get the project and install its packages

> **Important.** Do **not** run `npm create vite` or start a new project. The lead owns the skeleton (Vite, React Router, lib/, src/shared/). You work inside the shared repo.

1. The team repository already exists and you have access. Clone it (use the repository URL from GitHub, under the green Code button) and move into the folder.
2. Create your branch.
3. Install everything listed in the lead's `package.json`.
4. Check the stack is present.

```
cd C:\Projects
git clone <repo-url>
cd <repo-folder>
git checkout -b part-3-returns
npm install
npm ls mssql bcrypt jsonwebtoken jest react-router-dom vite
```

- If a package is missing, **tell the lead** instead of committing a change to `package.json`, so the five parts do not conflict. To keep testing locally in the meantime: `npm install mssql bcrypt jsonwebtoken react-router-dom` and `npm install --save-dev jest vite`.

- If `bcrypt` fails to install (it is a native module), the lead can switch to `bcryptjs`. Otherwise install "Desktop development with C++" from the Visual Studio Build Tools and retry.

Step 4: Let Node connect to your local SQL Express

VS Code's SQL extension can connect in ways Node's `mssql` package cannot, so this step is often the real blocker.

1. Open **SQL Server Configuration Manager**. Under SQL Server Network Configuration, Protocols for SQLEXPRESS, right-click **TCP/IP** and choose Enable.
2. Under SQL Server Services, set **SQL Server Browser** to Automatic and Running (needed to find the named instance).
3. Restart **SQL Server (SQLEXPRESS)** in the same list.
4. Node needs a SQL login, not Windows authentication. In VS Code, on `master`, check the mode. A result of 0 means mixed mode is already on. If it returns 1, enable "SQL Server and Windows Authentication mode" in the server Properties, Security page, then restart the service.

```
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly');
```

5. Create a local development login. This is for your machine only; choose your own password.

```
CREATE LOGIN pos_dev WITH PASSWORD = 'ChooseAStrongPassword!1';
GO
USE pos_store;
GO
CREATE USER pos_dev FOR LOGIN pos_dev;
ALTER ROLE db_owner ADD MEMBER pos_dev;
GO
```

6. Put your settings in `.env.local` in the repo root. Ask the lead for the exact variable names that `lib/db.js` reads, and make sure `.env*` is in `.gitignore`. **Never commit this file.**

Setting	Local value
Server	localhost
Instance	SQLEXPRESS
Database	pos_store
User / password	The pos_dev login you created
Encryption	encrypt false and trust server certificate true (local only; the Azure VM uses encrypt: true)

Step 5: Vercel CLI (optional)

It runs the `api/` functions locally the way they run in production. The functions are plain Node handlers, not Express.

1. Install and log in: `npm install -g vercel`, then `vercel login`.
2. The lead must first add you to the Vercel project. Then run `vercel link` inside the repo.

3. Do **not** pull production environment variables unless the lead says so, because that downloads real credentials. Use your own `.env.local`.
4. Run `vercel dev` for app and API together. If the lead's setup uses `npm run dev` for the front end only, use that for screen work and rely on Jest for the API.

Step 6: Optional VS Code extensions

- **ESLint** and **Prettier**, if the repo uses them.
- **Thunder Client** or **REST Client**, to call `api/returns` without a screen.

Step 7: Verify the setup

[ ]	`git --version`, `node -v` and `npm -v` each print a version.
[ ]	`git branch` shows `* part-3-returns`.
[ ]	`npm ls` shows no missing packages.
[ ]	Nested triggers are on: `SELECT value_in_use FROM sys.configurations WHERE name = 'nested triggers';` returns **1**. The triggers chain (your refund triggers feed the loyalty and low-stock triggers), so this is required.
[ ]	`SELECT COUNT(*) FROM sys.tables;` returns 27, and `SELECT name FROM sys.triggers;` lists at least the 5 built triggers (your own two will be added to the list). If the counts are off, rebuild the database from the lead's files in the order in the schema doc: schema file, then `triggers.sql`, each exactly once.
[ ]	A small Node script (or the lead's `query()` from `lib/db.js`, if it exists) connects as `pos_dev` and runs `SELECT 1`.
[ ]	`npx jest` runs without crashing ("no tests found" is fine).
[ ]	`npm run dev` starts Vite and the page opens in the browser.

> **Troubleshooting.** If the connection test fails, the usual causes are TCP/IP disabled, SQL Server Browser stopped, mixed mode off, or the service not restarted. Send the exact error message when asking for help.

4. What you need from the lead

[ ]	Vercel project access, if you want to use the CLI (you already have access to the repository).
[ ]	`lib/db.js` (`query(sql, params)` and `withTransaction(fn)`), `lib/auth.js` (`requireAuth(['manager'])(handler)`) and `lib/http.js` (`sendJson`, `sendError`). Names are proposed and became final at the end of Day 2, so confirm they are merged.
[ ]	`src/shared/` components and the API client (`api.get`, `api.post`), plus login and register, and the employee shell, so you can test as a customer, as support and as a manager.
[ ]	The **seed script**: products with stock, customers, employees (at least one Support and one Manager login) and **completed sales** to return against, including one sale older than 30 days for the window test, exactly one reward program with tiers, and a TAX_RATE for every category in every test state.
[ ]	The exact variable names for `.env.local`, and a least-privilege database login for the Azure VM (not sa) when you need it.
[ ]	The schema file, `triggers.sql`, `test_triggers.sql` and `drop.sql`. They are not attached to this guide. The schema doc contains the text of the 5 built triggers.
[ ]	Whether Part 2's checkout is merged yet. Until it is, test against the seed script's completed sales. Part 2's order detail page links to `/returns/new?order=<id>`, and that route is yours.

5. Rules you must follow

Database rules (shared by all parts)

1. Triggers are AFTER, set-based and named `TR_<Table>_<Purpose>`. They start with `SET NOCOUNT ON; SET XACT_ABORT ON;` and fail with `THROW`. Every SQL file begins with `USE pos_store;` then `GO`.
2. Never edit `CUSTOMER.Loyalty_points` or `Tier_id` directly. Points come from a LOYALTY_TRANSACTION row, and the built trigger updates the rest.
3. Never recompute stock, points, tier or low-stock alerts in the app. The built triggers keep them in sync.
4. Every query is parameterized. Prices, tax and totals come from the database, never from the request.
5. A write that touches several tables runs in one database transaction, using the lead's helper.
6. Each trigger's tests go in `db/returns/test.sql`. Every failing case runs in its own transaction and rolls back.

Decisions already closed (do not reopen)

Topic	Decision
Refund formula	Refunds include proportional tax. Per returned line: refund = round((L + T * L / S) * q / n, 2). L is the line total of the product, T the sale's Tax_amount, S its Subtotal, q the returned quantity and n the purchased quantity. Refund_amount must not exceed it.
Restock	Returned items go back to the **same product** at its original condition: the trigger adds q to that product's Stock_quantity. That fires the low-stock trigger, which may close an open alert.
Return window	**30 days** from the sale's Transaction_date. The database enforces it in TR_PRODUCT_RETURN_ITEM_Guard by comparing the return's Requested_at with the sale's Transaction_date.
Status flow	A return moves requested, then approved or denied, then refunded. Items lock once the return leaves 'requested'.
Handler	PRODUCT_RETURN.Employee_id is nullable: a customer requests first, staff handle later. The handler's id is saved at approval or denial and always comes from the login token. CK_PRODUCT_RETURN_Handler: Status = 'requested' OR Employee_id IS NOT NULL.
Loyalty points	Earn only, never spent. A refund inserts a negative LOYALTY_TRANSACTION of FLOOR(L * q / n * Points_per_dollar), because points were earned on the Subtotal and not on tax. Trigger 2 rejects a reversal larger than the current balance, so the app decides how to handle that edge case.
Totals	Subtotal = SUM(Line_total) (after discount, before tax); Total = Subtotal + Tax_amount. The refund formula relies on this.
Customers and roles	No guest checkout; the customer id always comes from the login token. Customers create and list their own returns. Support and managers approve, deny and refund. The rewards report is for managers only.

Why your numbers matter downstream. Part 4's alert-to-restock flow relies on your restock firing the low-stock trigger, and the loyalty chain relies on your negative row carrying the right Program_id (section 6). If these are wrong, stock and points drift silently.

6. The data you work with

Figure 1, on the last page of this guide, draws these tables and how they connect. Read the table below for the details of each column, then use the figure to see the relationships. Teal tables are the ones you write, navy tables you only read, and amber tables are changed by triggers (never write them from the app).

Table	What Part 3 needs to know
PRODUCT_RETURN	Return_id, Transaction_id (the sale), Employee_id (nullable: the handler), Requested_at, Status ('requested', 'approved', 'denied', 'refunded'), Reason, Refund_total, Refunded_at. **There is no Customer_id:** find a customer's returns through the sale (SALES_TRANSACTION.Customer_id). Approve and deny updates must set Employee_id in the same statement or CK_PRODUCT_RETURN_Handler rejects them.
PRODUCT_RETURN_ITEM	PK (Return_id, Product_id): one row per product per return. Quantity (> 0), Refund_amount (>= 0). Foreign keys to PRODUCT_RETURN and PRODUCT.
SALES_TRANSACTION	Read only for you. Customer_id (NOT NULL, the owner), Transaction_date (start of the 30 days), Subtotal (S), Tax_amount (T), Status (only a 'completed' sale can be returned).
TRANSACTION_ITEM	PK (Transaction_id, Product_id). Quantity (n, purchased), Unit_price, Discount_amount, Line_total (L: persisted computed column, after discount and before tax).
CUSTOMER	Loyalty_points and Tier_id are set only by the loyalty trigger. Read them for the rewards page; never update them.
LOYALTY_TRANSACTION	Loyalty_txn_id, Customer_id, Transaction_id (nullable, SET NULL), Program_id, Points_change, Reason (free text, up to 100 characters), Created_at. Earn rows have Reason 'Purchase'. Your refund trigger inserts the negative row. The rewards history reads this table.
REWARD_PROGRAM / REWARD_TIER	REWARD_PROGRAM has Points_per_dollar, Effective_from and Effective_to (NULL = no end); no two programs overlap. REWARD_TIER has Tier_name, Min_points (unique per program) and Perk_description. The next tier is the lowest Min_points above the balance in the same program.
PRODUCT	Stock_quantity is increased by your refund trigger, on the same product. Products with sales history are deactivated, not deleted, so Is_active can be 0 on an old sale's product.
LOW_STOCK_ALERT	Not yours. TR_PRODUCT_LowStock closes an open alert when your restock lifts stock above the threshold.
EMPLOYEE	Role is 'Manager' or 'Support'. The handler's Employee_id comes from the login token.

What already happens (do not rebuild any of it)

1. When a sale is completed, **TR_SALES_TRANSACTION_Complete** subtracts stock and inserts a LOYALTY_TRANSACTION for FLOOR(Subtotal * Points_per_dollar), using the reward program active on the sale date. Your refund trigger is the mirror of this.
2. A stock change fires **TR_PRODUCT_LowStock**: it opens an alert at or below the threshold and closes the open alert once stock is above it.

3. A loyalty row fires **TR_LOYALTY_TRANSACTION_Balance**: it adds Points_change to the customer's balance (error 50001 if the balance would go negative), then sets Tier_id to the highest tier whose Min_points is at or below the balance **in the Program_id of the inserted row**. It already works for negative rows.

So your refund update fires a chain: `TR_PRODUCT_RETURN_Refunded` changes stock and inserts a negative loyalty row, which fire the two triggers above. If any link fails (for example the balance check), XACT_ABORT rolls back the whole refund.

7. Build order

Do the SQL first, then the API, then the screens. Your SQL files are due to the lead tomorrow (Oct 21).

Phase 0: Sync with the lead (before writing code)

- Confirm the lib/ helper names, the seed script and your Support and Manager logins, and go through the questions in section 10 (including which THROW numbers you may use: 50001 and 50002 are taken).
- Tell the lead early if Oct 21 is not realistic for the two triggers and their tests.

Phase 1: TR_PRODUCT_RETURN_ITEM_Guard (db/returns/triggers.sql)

AFTER INSERT and UPDATE on PRODUCT_RETURN_ITEM. It rejects the statement unless **all** of these hold:

Rule	Detail
Sale is valid	The return's original sale is 'completed' and contains the product.
Within 30 days	The return's Requested_at is within 30 days of the sale's Transaction_date.
Return still open	The parent return is still 'requested'. Items lock after approval, denial or refund.
Quantity	Returned quantity summed over all non-denied returns of that sale and product is at most the purchased quantity.
Refund amount	Refund_amount is at most the formula: round((L + T * L / S) * q / n, 2).

```
refund = round( (L + T * L / S) * q / n , 2 )
```

- Handle multi-row statements through `inserted`, and use `triggers.sql` as the style model. On an UPDATE, q is the new quantity.
- **Example (made-up numbers, for your tests):** sale Subtotal S = 100.00, Tax_amount T = 8.25, a line with L = 40.00 for n = 2 units. Returning q = 1 gives round((40.00 + 8.25 * 40.00 / 100.00) * 1 / 2, 2) = 21.65, so 21.65 passes and 21.66 is rejected. With 2 points per dollar the reversal is FLOOR(40.00 * 1 / 2 * 2) = 40 points.
- The guard does **not** check who owns the sale. That check belongs in the API: the sale must belong to the customer in the token.

Phase 2: TR_PRODUCT_RETURN_Refunded

AFTER UPDATE of Status on PRODUCT_RETURN. When a return becomes 'refunded' it does three things:

1. Sets Refunded_at.

2. Restocks: adds each returned item's quantity to Stock_quantity of the same product.
3. Inserts a negative LOYALTY_TRANSACTION for the points earned on the refunded amount: FLOOR(L * q / n * Points_per_dollar), as a negative Points_change.

- Mirror TR_SALES_TRANSACTION_Complete: act only on rows that just changed to 'refunded' from another status, so re-saving a refunded return does not restock or reverse twice.
- The negative row's Program_id must be the reward program active on the **sale's** date (the same one used to earn the points), because trigger 2 sets the tier from the Program_id of the inserted row. This is a suggestion that follows from how trigger 1 and trigger 2 work; test it.
- Do not touch LOW_STOCK_ALERT, CUSTOMER.Loyalty_points or Tier_id: the built triggers do that.

Decide and write down before coding

These details are not fixed in the briefs. The suggestions are mine, not decisions: confirm the ones that affect others with the lead.

- **Refund_total**: who sets it and when? Suggestion: the API sets it as the sum of the items' Refund_amount when the request is created.
- Should the refund trigger reject a return that becomes 'refunded' from any status other than 'approved'? No trigger enforces the status order today, so the API must send only valid transitions.
- Points reversal: one negative row per return (suggestion), and is it the sum of each line's FLOOR value or FLOOR of the sum? Pick one, write it down and test it. Skip the row when the result is 0, as trigger 1 does. Choose the Reason text (for example 'Return refund').
- What the app shows when the refund fails because the reversal is larger than the balance (error 50001 rolls back the whole refund).
- The formula divides by S: what if the sale's Subtotal is 0 (everything discounted)? Suggestion: treat the tax share as 0. Also use a higher-precision decimal before dividing, round only at the end, and compare with a hand-calculated value in your test.
- Returning a product that is now inactive (Is_active = 0): the briefs do not mention it; suggestion: allow it. The request body is the order id, the items (product id and quantity) and a reason: no amounts and no customer id.

Phase 3: Tests (db/returns/test.sql)

Each failing case runs in **its own transaction** that is rolled back, because a failing statement leaves the transaction unusable. Build your own test data like `test_triggers.sql` does (customer, product, a completed sale). A backdated sale needs a reward program whose Effective_from covers that date. If Part 2's payment triggers are already in your database, insert PAYMENT rows (equal to Total) before completing a test sale; doing it anyway is harmless.

Required test	Expected result
Return requested more than 30 days after the sale	Rejected by TR_PRODUCT_RETURN_ITEM_Guard
Return more than was purchased	Rejected by TR_PRODUCT_RETURN_ITEM_Guard
Refund_amount above the formula	Rejected by TR_PRODUCT_RETURN_ITEM_Guard
Change an item after the return was approved	Rejected by TR_PRODUCT_RETURN_ITEM_Guard
Mark a valid return refunded (happy path)	Stock goes up on the same product, Refunded_at is set, a negative loyalty row is inserted, the customer's points and tier update

Recommended extras: a sale that is not completed; a product not in the sale; two returns that together pass the purchased quantity (a denied one does not count); a multi-row insert; re-saving 'refunded' does not restock twice; cent rounding; a reversal larger than the balance. When done, send `triggers.sql` and `test.sql` to the lead.

Phase 4: Returns API and screens

- `api/returns/`: customers create a request (return and items inserted in one transaction, status 'requested', Employee_id NULL) and list their own returns. Support and managers see the queue of requested returns and approve, deny or mark refunded. Customer id and handler id come from the token. Check in the API that the sale belongs to the customer.
- Approve and deny save the handler's Employee_id in the same update. Refunding is the status update to 'refunded': never touch stock or points yourself.
- Jest tests that call the handlers directly, including a customer returning another customer's sale, a customer approving, and trigger errors coming back as readable messages.
- Screens: **request a return** from an order (items, quantities, reason; the route `/returns/new?order=<id>` that Part 2's order detail links to), **my returns**, and the **support returns queue**. Each handles loading, empty and error states.

Phase 5: Rewards API and page

- `api/rewards/`: the customer's points balance, tier, next tier and history (CUSTOMER, LOYALTY_TRANSACTION, REWARD_TIER), with a Jest test.
- A rewards page showing points, tier and history. The tier progress bar is cut-first.

Phase 6: Rewards report

- `api/reports/rewards.js`, managers only: points issued and reversed, customers per tier, and top earners (cut-first). A Jest test, including a check that a non-manager is refused.
- The report page in `src/pages/reports/` (one file, added only by you).

Phase 7: Finish

Run your triggers and tests together with the lead's chain (a refund restocks, updates an open low-stock alert and reverses points), then open a pull request from `part-3-returns` to the lead.

8. Schedule, pacing and cut order

Date	Plan milestone	Suggested pacing for Part 3
Oct 20 (Day 5, today)	Parts 1 to 4 build (Oct 17 to 22)	Install, read the docs, Phase 0, start Phase 1
Oct 21 (Day 6)	**Your triggers.sql and test.sql due to the lead**	Finish Phases 1 to 3 and send the files
Oct 22 (Day 7)	Lead runs the full chain through the app	Phase 4: returns API and its three screens
Oct 23 (Day 8)	**Feature freeze**	Phases 5 and 6, then Phase 7 and the pull request
Oct 23 to 24	Test and bug bash	Fix bugs found by the team
Oct 25 (Day 10)	Submission	One .bak or .sql file (schema, then triggers)

The pacing column is a suggestion; the milestones come from the project plan. The schema is frozen: ask the lead before changing tables.

If you fall behind, cut in this order

1) Next-tier progress and the tier progress bar. 2) Top earners in the rewards report. 3) The return reasons list. Never cut the two triggers, the refund with restock and points reversal, or the rewards report.

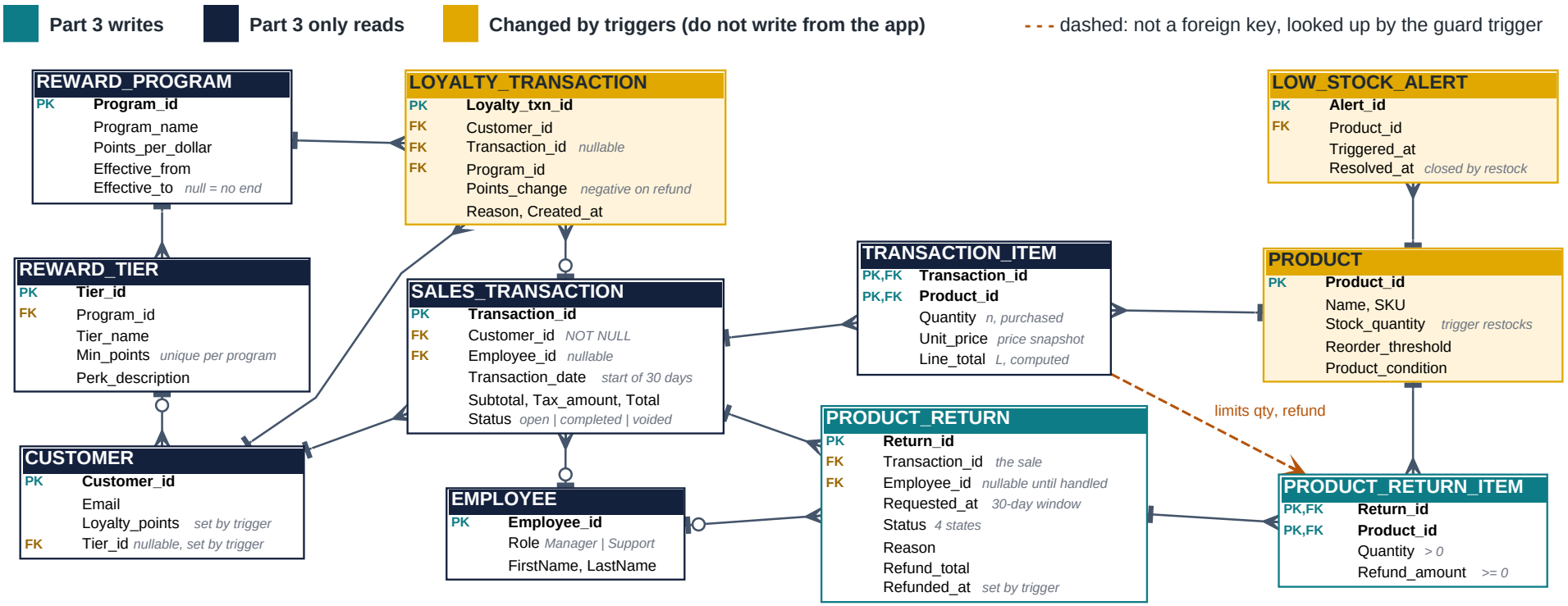
9. Definition of done

- [ ] Both triggers are built, named correctly, set-based, and the five required SQL tests pass in `db/returns/test.sql`, each failing case in its own rolled-back transaction.
- [ ] A valid refund restocks the same product, sets Refunded_at and reverses the points, and the customer's points and tier update through the built triggers.
- [ ] Each endpoint (returns, rewards, rewards report) has a Jest test that calls the handler directly. A customer cannot return another customer's sale or approve a return, and the rewards report refuses non-managers.
- [ ] Each screen handles loading, empty and error states.
- [ ] The route `/returns/new?order=<id>` works from Part 2's order detail page.
- [ ] The app never writes stock, Loyalty_points or Tier_id, and no secrets are committed.
- [ ] The pull request is open against the lead.

10. Questions to confirm with the lead

- Which THROW error numbers may your triggers use (50001 and 50002 are taken)?
- The exact variable names `lib/db.js` reads, whether `lib/` and the seed script are merged, whether the seed has a completed sale older than 30 days, whether Part 2's checkout is merged, and how the Azure login will be shared.
- The guard is listed for INSERT and UPDATE only: should deleting an item after approval also be blocked (add DELETE), or does the API block it?
- Does `PRODUCT_RETURN.Requested_at` have a database default, and who sets Refund_total (section 7)?
- Should a reversal larger than the balance (error 50001) stop the refund with a clear error? Should the rewards report cover all time or a date range?

Figure 1. The tables behind returns and rewards and how they connect



Reading the lines. A bar means one parent row, a crow's foot means many child rows, and a circle means optional (the foreign key can be NULL). Solid lines are foreign keys. The one dashed line is not a foreign key: the guard trigger finds the sale's line in TRANSACTION_ITEM through the return's Transaction_id and the item's Product_id, to get n and L and to cap the quantity and the refund.

Returns have no customer. PRODUCT_RETURN points only to the sale, so a customer's returns are found through SALES_TRANSACTION.Customer_id, and the customer id comes from the token. The guard trigger does not check ownership: the API must.

What happens when a return becomes 'refunded'. Your TR_PRODUCT_RETURN_Refunded adds the returned quantity to PRODUCT.Stock_quantity and inserts a negative LOYALTY_TRANSACTION. TR_PRODUCT_LowStock then closes any open LOW_STOCK_ALERT, and TR_LOYALTY_TRANSACTION_Balance updates CUSTOMER.Loyalty_points and Tier_id. You write only the teal tables from the app.

Not drawn. TRANSACTION_ITEM also has Discount_id and Discount_amount (DISCOUNT is not drawn), and LOW_STOCK_ALERT has an optional Order_id to SUPPLIER_ORDER. Part 3 does not touch DISCOUNT or the supplier tables.